

Web application with user registration and login – Vili Sihvo

This project is a secure full-stack web application featuring a main interface (MainPage) with user authentication features, register, login and logout. The application emphasizes security practices by safely handling credentials, sessions, data transfers and storing, and request flows throughout the system. GitHub is used for version control and publishing the updates.

The frontend is built with React as a single page application, using Axios to communicate with the backend through HTTP requests and API interactions. The backend is implemented in JavaScript, using Express with SQLite3 as a database. The application is served through Nginx reverse proxy which delivers all the React frontend routes all /api requests to the backend. The entire system is fully Dockerized and composed with Docker Compose which enables for easy usage across different machines.

To monitor and validate the state of the application, some tests have been implemented for both frontend (vitest) and backend (jest) and the CI pipeline built for automated testing contains a lot of dynamic and static testing tools and aids.

Motivation for the application came from interest and desire to build a safe user authentication system since I've never constructed one. Understanding how such a system is designed and implemented has been a mystery for me so not everything is perfect and this has been a learning process to try out new things and understand concepts of how to do things in different scenarios.

Instructions

Prerequisites: Docker

1. Download the repository directory somewhere
2. Open CMD/bash etc and navigate to downloaded directory root
3. Change directory to backend and create .env file
4. Set these environmental variables to the .env file

```
JWT_SECRET=koira666
NODE_ENV=development
PORT=5000
CLIENT_URL=http://localhost:5173
DOCKER_CLIENT_URL=http://localhost:8080
REDIS_URL=redis://redis:6379
ACCESS_SECRET=some_super_secret_access
REFRESH_SECRET=some_super_secret_refresh
SESSION_SECRET=some_super_secret_session
```

5. Change directory back to root and run the Docker Compose with command:

```
docker compose up --build
```

6. Wait for all the services to start and run. The site can be accessed on URL localhost:8080.

Structure of the program

Application follows modular, containerized full-stack architecture with clear separation between frontend and backend. Frontend contains the UI, its logic, client-side validation, state management Nginx configuration as well as API for communication. Backend includes the concrete server, database, validators and utilities for the logic as well as middleware for authenticating purposes.

Each module has been commented in detail in their own files and more information about them can be found in corresponding files.

This is the structure of the program directory. Some files (node_modules, coverage, .env, .dockerignore) have been left out for easier examination.

```
.
├── .github \ workflows
|   ├── sast.yml           workflow GitHub Actions
├── README.md
├── backend
|   ├── Dockerfile         Backend Docker container build instructions
|   ├── controllers
|   |   └── authController.js Login/register/logout etc. logic
|   ├── database.js        Database initialization + connection
|   ├── middleware
|   |   ├── authAccessToken.js JWT access token verification
|   |   ├── csrfMiddleware.js  CSRF protection middleware
|   |   └── rateLimit.js       Rate limiting rules
|   ├── package-lock.json
|   ├── package.json
|   ├── routes
|   |   ├── authRoutes.js     /api/auth routes
|   |   ├── csrf.js           CSRF token GET route
|   |   └── userRoutes.js      protected user routes
|   ├── server.js           Express server setup + Redis sessions
|   ├── services
|   |   └── authService.js     Auth-related DB/service logic
|   └── tests
```

- | | | └─ authAccessToken.test.js
- | | | └─ authController.test.js
- | | └─ server.test.js
- | └─ utils
- | | └─ csrf.js CSRF token utilities
- | | └─ logger.js Log/printing utilities
- | | └─ validation.js Input validation helpers
- | └─ docker-compose.yml Multi-container setup (frontend, backend, redis, nginx)
- | └─ frontend
- | | └─ Dockerfile Frontend Docker container build instructions
- | | └─ index.html Root html
- | | └─ nginx.conf Nginx config (proxy)
- | | └─ package-lock.json
- | | └─ package.json
- | | └─ src
- | | | └─ App.jsx Root react component, “main”
- | | | └─ components
- | | | | └─ ProtectedRoute.jsx Route guard for authenticated page(s)
- | | | | └─ main.jsx React entry point
- | | | | └─ pages
- | | | | | └─ LoginPage.jsx Login UI
- | | | | | └─ MainPage.jsx Main authenticated UI
- | | | | | └─ RegisterPage.jsx Registration UI
- | | | └─ redux
- | | | | └─ authSlice.js Auth state management
- | | | | | └─ store.js Redux store setup
- | | | | └─ services
- | | | | | └─ api.js Axios instance + base config
- | | | | | └─ authService.js Frontend auth API calls
- | | | └─ style.css

- | | └─ utils
- | | └─ registerSchema.js Registration validation schema (input rules)
- | | └─ logger.js Frontend logging/printing
- | └─ tests
- | | └─ LoginPage.test.jsx
- | | └─ MainPage.test.jsx
- | | └─ ProtectedRoute.test.jsx
- | | └─ RegisterPage.test.jsx
- | | └─ api.test.js
- | | └─ authService.test.js
- | | └─ authSlice.test.js
- | | └─ setupTests.js
- | └─ vitest.config.js
- └─ package-lock.json
- └─ package.json
- └─ sonar-project.properties

Backend structure:

- controllers/
 - Handles request-response logic (register, login, refresh etc.)
- services/
 - Handles authentication services and database operations etc
- routes/
 - Defines API endpoints such as /api/auth & /api/user
- middleware/
 - Security and request-processing middleware including
 - JWT authentication middleware
 - CSRF protection
 - Rate limiting
- utils/
 - Shared utility modules:
 - Token generation and validation
 - Password hashing and validation
 - CSRF helpers
 - Logging utilities
 - Input validation schemas
- tests/
 - Unit and integration testing
- Core backend files:

- server.js
 - Express application setup, middleware conf, Redis session
- database.js
 - SQLite database initialization and connection logic

Frontend structure:

- src:
 - pages/
 - LoginPage.jsx
 - User login interface
 - RegisterPage.jsx
 - User registration interface with client-side validation
 - MainPage.jsx
 - Protected authenticated user area
 - components/
 - ProtectedRoute.jsx
 - Route guard enforcing authentication checks
 - services/
 - api.js
 - Axios configuration with interceptors for JWT and CSRF handling
 - Usual API responsibilities
 - authService.js
 - API abstraction layer for authentication requests
 - utils/
 - Logging utility
 - tests/
 - Tests for the frontend
 - main.jsx/App.jsx
 - Application entry point and root component

The whole system is fully containerized using Docker.

Docker-compose.yml contains:

- Backend service
- Frontend service (served via nginx)
- Redis service
- Internal Docker networking between services

Nginx configuration:

- Server frontend static build
- Acts as reverse proxy for /api/ requests to backend
- Ensures backend is not directly exposed externally

CI/CD pipeline (.github/workflows/sast.yml)

- Automated testing and builds
- Dependency scanning (Snyk, Trivy)
- Static analysis (SonarQube)
- Security testing (OWASP ZAP)

- SBOM generation (CycloneDX)
- Runs on push and pull requests
- Self-hosted

.env File contents

JWT_SECRET=koira666

NODE_ENV=development

PORT=5000

CLIENT_URL=http://localhost:5173

DOCKER_CLIENT_URL=http://localhost:8080

REDIS_URL=redis://redis:6379

ACCESS_SECRET=some_super_secret_access

REFRESH_SECRET=some_super_secret_refresh

SESSION_SECRET=some_super_secret_session

Security-Oriented design

The architecture is designed with security first in mind.

- Authentication uses JWT access tokens and refresh tokens with safe properties nonHttp, sameSite and secure.
- Session management with Redis sessions
- CSRF tokens (submit token pattern)
- Input validation both on frontend (Yup/Formik) and backend (Zod)
- Database queries use parametrized statements to prevent SQL injections
- Containers isolate services
- Nginx ensures controlled external exposure

Secure programming solutions

Registration flow

Register – frontend

- Rate-limited, Brute-force prevention
- Username
 - o 3-40 characters
 - o Only letters, numbers or underscores
 - o unique
 - o Required
- Email
 - o Email format ([x@y.z](#))
 - o unique

- required
- Password
 - Minimum 12 characters
 - Must have at least one uppercase character, lowercase character, number and a special character
 - Must be repeated to prevent typos
 - required
 - Error messages don't leak too much information – if username or email is not unique "Username or email already used" will be shown as an error to not give out any credentials to potential abusers
- Uses Formik + Yup validation
- Prevents invalid requests from reaching backend

API Layer

- Accepts /api/auth/register request and sends it to backend

authController – backend

- Validates incoming request using backend schema

Backend validation – utils/validation.js ← Zod

- Enforces strict server-side validation rules (same than in frontend)
 - Username
 - Email
 - Password
- Final security gate

Service layer (authService.js)

- Checks for existing username/email using parametrized SQL query.
- Hashes password `bcrypt.hash(password, 12)`
- Generates secure user id `crypto.randomUUID()`

Database layer (database.js, db)

- Stores only hashed passwords

After successful registration client is redirected to login.

Security point of view

- Prevents SQL injection via parametrized queries
- Forces safe passwords and typo prevention
- Generic error messages
- Brute force protected for misuse and to see which email/username is used
- Validation in frontend and backend

Login flow

Login – frontend

- Rate-limited, Brute-force prevention
- Error message doesn't leak information – only Invalid credentials on invalid username or password, else Login failed
 - o Can't abuse to check which username is used
- Login form collects username and password
- Input is sent to backend with api.js (centralized Axios instance)
- Request automatically attaches:
 - o CSRF token (if required ← currently only for logout)
 - o HTTP-only session credentials (withCredentials: true)

Backend API layer (routes/authRoutes.js)

- Defines /api/auth/login endpoint
- Forwards request to controller layer only
- No business logic is executed in routes

Controller layer (controllers/authController.js)

- Handles request validation flow
- Calls authentication service
- Converts service output to HTTP response

Service layer (services/authService.js)

- Core authentication logic:
 - o Fetches users from database using parametrized query
 - o Verifies password using bcrypt.compare()
 - o Generates JWT access + refresh tokens

After successful login, JWT tokens are issued, user state is stored in application state user is redirected to main page.

Security point of view

- Brute-force protection enabled
- Safe error responses
- Input validation on frontend and backend
- User authentication safely

Main view / MainPage.jsx

Main page represents the authenticated application area. It's accessible after successful login and token verification. All API requests initiated from this page use Axios that automatically attaches the access token in authorization header and includes CSRF token for state changing request. For now, the logout feature is the only one used and featured in the application. Logging out destroys the session, clears cookies and redirects the user back to login page.

Main page – frontend

- Access is restricted using a Protected Route component
- Users must have a valid access token to access this page
- Unauthorized users are redirected to /login

Security:

- Before rendering MainPage, system verifies for valid access token
- Main page only accessible after successfully authenticated
- Logout is POST and it needs a CSRF token

CI/CD pipeline

Project uses a self-hosted GitHub actions CI pipeline. The workflow is triggered on every push and pull request to the main branch ensuring that all changes are verified before integration. These are the pipeline stages:

1. Source control & build preparation
 - a. Repository checkout and node configuration
2. Dependency installation
 - a. separately for root, frontend and backend
3. All tests are ran with coverage
4. Backend .env file is created for testing and testing secrets are injected there
5. Docker images are built and the compose started
6. Timer for application startup (sleeps 45 secs)
7. OWASP ZAP scan for security threats and its report artifact
8. Snyk scan for vulnerable or outdated dependencies ← there's a premade line commented out for stricter production type limit which would result in a fail if a high or critical warning would appear
9. Trivy scans frontend and backend images separately for Docker image vulnerabilities
10. CycloneDX scans supply chain vulnerabilities and provides an artifact
11. SonarQube analyses code quality and security vulnerabilities statically ← results updated to their dashboard
12. Stop docker compose

OWASP TOP 10

A01:2025 Broken Access Control

- Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
 - o The application does not contain roles
 - o Authenticated users can only access protected routes via middleware
 - o Actions and access are available for users but are not granted for particular roles etc
 - authentication state (JWT + session)
- Bypassing access control checks by modifying the URL, internal application state, or the HTML page, or by using an attack tool that modifies API requests
 - o Backend routes such as /api/user/* require valid JWT access tokens
 - o Axios automatically attaches Bearer tokens via request interceptor
 - o Unauthorized requests are rejected at middleware level before reaching controllers

- Permitting viewing or editing someone else's account by providing its unique identifier
 - o User ID is assigned safely for user on the backend, and it can't be guessed as it's generated with randomUUID()
 - o Routes involve the access token checks for actions
 - o User identity is derived from JWT payload (id, username)
 - o Database queries use server-side resolved identifiers
- API accessible with missing access controls for POST, PUT, and DELETE
 - o API has GET endpoints for CSRF-token fetch which is "public" route and don't require access or CSRF -token
 - o Currently CSRF + access token are required for logging out since there's not activities to do when logged in
 - o Non-state-changing POST actions for the server side which are user-oriented starting from the input like login and register are considered relatively safe according to some googling and thus don't require CSRF. However the application has allowedOrigins set and uses safe measures all over so this wont be a problem.
- Elevation of privilege. Acting as a user without being logged in or gaining privileges beyond those expected of the logged in user
 - o Access token authenticating prevents user impersonating and since there are no roles no admin-level etc privileges can be achieved
 - o Access token expiration limits long-term misuse
- Metadata manipulation, such as replaying or tampering with a JWT access control token, a cookie or a hidden field manipulated to elevate privileges, or abusing JWT invalidation.
 - o Server uses server sessions (express-session + redis) which allows that sessions can be invalidated server-side and centralized control
 - o Cookies are httpOnly which prevents XSS token stealing
 - o JWT tokens are signed server-side using secrets
 - o STP-type CSRF-token pattern protects cookie-based auth endpoints from cross site abuse.
 - o Access token expires in 15 minutes & refresh token in 7 days
- CORS misconfiguration allows API access from unauthorized or untrusted origins
 - o CORS is configured to restrict the origin to be the frontend and to require credentials
 - o However, I believe CORS does not prevent submitting forms from other URLs but those are taken care of with CSRF and access token requirements
- Force browsing (guessing URLs) to authenticate pages as unauthenticated user to privileged pages as a standard user
 - o If one was to guess a correct URL, the middleware still prevents these by requiring a valid access token which is authenticated

A02:2025 Security Misconfiguration

- It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services.
 - o Helmet.js is used to handle Headers securely
 - o Adds secure HTTP response headers automatically
 - o Prevents MIME/content-sniffing
 - o Applies security policies
 - o Secure cookie settings are used (httpOnly, sameSite, secure)
 - o Environment variables are used for secrets and configuration
 - o Redis session storage is used instead of insecure default memory storage

- Docker containers are used to isolate backend, frontend and Redis services
 - Separate Docker network is configured
 - Environmental variables are used instead of hardcoding or exposing secrets
 - Redis connection is configured through internal Docker networking
 - Persistent backend storage is isolated using it as a volume
 - Nginx is used as reverse proxy to control external access
 - Only nginx is exposed externally while internal services are “isolated”
 - Frontend Axios automatically attaches authentication and CSRF headers
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
 - Development logging is restricted to development mode only (some for testing)
 - Redis is disabled during automated testing
 - Only required API routes are exposed (/auth, /user, /csrf-token)
 - Rate limiting is applied to reduce abuse (brute forcing and guessing etc)
 - Services are separated into dedicated containers (backend, frontend, Redis)
 - Internal services communicate through Docker network
 - Only required ports are exposed externally
- Default accounts and their passwords are still enabled and unchanged.
 - Session secrets are configured using env variables
 - Application startup fails if SESSION_SECRET is missing
 - No default authentication credentials are used and abusable URLs, ports and variables are also env variables
- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
 - Centralized Express error handling
 - Generic error messages are returned to clients
 - Internal stack traces are not exposed to users
 - Debug logging is separated from production behaviour (checks if it is running in dev, testing or production mode)
- For upgraded systems, the latest security features are disabled or not configured securely.
 - Helmet enables modern security headers
 - Secure cookie policies are configured
 - CORS restrictions are configured explicitly
 - Session handling uses secure configurations practices
- Excessive prioritization of backward compatibility leading to insecure configuration.
 - CORS origins are restricted to “same site”
 - Only required HTTP methods are allowed
 - Explicit allowed headers are configured
 - Session cookies use modern security settings
- The security settings in the application servers, application frameworks (e.g., Struts, Spring, ASP.NET), libraries, databases, etc., are not set to secure values.
 - Express sessions are configured securely
 - Cookies use httpOnly and sameSite
 - CORS is restricted to the trusted frontend origin
 - Rate limiting
 - Redis session storage is configured securely
 - Docker networking is used to isolate application services
 - Redis uses internal container communication through Docker

- Application configuration is centralized with Docker Compose
- Persistent application data is separated using Docker volumes
- Nginx is used as a reverse proxy in front of front and backend services
- Backend service is not directly exposed to the internet
- ALL API traffic is routed through /api/ via nginx
- The server does not send security headers or directives, or they are not set to secure values.
 - Helmet configures secure HTTP headers automatically
 - Security-related browser directives are enabled
 - HTTP header hardening

A03:2025 Software Supply Chain Failures

- You do not carefully track the versions of all components that you use (both client-side and server-side). This includes components you directly use as well as nested (transitive) dependencies
 - NPM is used as a package manager and it provides package(-lock).json files to store dependencies in their responding paths as well as other directory related information
 - Docker images are versioned using tags
 - Dependencies are installed consistently in CI using npm install in all service directories
 - CycloneDX SBOM is used to document all dependencies and information about them
- The software is vulnerable, unsupported, or out of date. This includes the OS, web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries.
 - CI pipeline includes automated scanning
 - testing and coverage
 - SonarQube – static analysis
 - Snyk – Dependency scanning
 - Trivy – Docker Container scanning
 - CycloneDX – Supply chain scanning
 - OWASP ZAP – Runtime security
 - Docker images use maintained base images
 - CI pipeline runs on every push and pull request
 - GitHub actions
 - Self-hosted (for experimental and learning purposes)
 - Contains secrets which are injected from Action secrets
- You do not scan for vulnerabilities regularly and subscribe to security bulletins related to the components you use.
 - Pipeline mentioned above automatically runs on every push and pull request to main
 - Snyk is used for dependency vulnerability scanning and monitoring
 - Trivy scans container images for recognized vulnerabilities
 - OWASP ZAP performs automated runtime security tests
 - SonarQube is used for static analysis of code quality and security issues and also for monitoring them easily via its dashboard.
- You do not have a change management process or tracking of changes within your supply chain, including tracking IDEs, IDE extensions and updates, changes to your organization's code repository, sandboxes, image and library repositories, the way artifacts are created and stored, etc. Every part of your supply chain should be documented, especially changes.
 - GitHub actions CI pipeline tracks all changes
 - All builds are reproducible with Docker Compose

- Dependencies are installed consistently
- Repository is currently private and artifacts are stored in the workflow runs which can be downloaded as zip files. If possible, the workflow runs with their details will be set to private.
- The tools used in the pipeline allow easy change tracking for different areas of the product, as well as the logging of the CI.
- Environmental variables are injected via secrets for controlled configuration management
- You have not hardened every part of your supply chain, with a special focus on access control and the application of least privilege.
 - Uses self-hosted runner (not so secure) ← should be changed to GitHub-hosted runner
 - CI pipeline could be split to test, scan, build pipelines and not one pipeline controlled
- Your supply chain systems do not have any separation of duty. No single person should be able to write code and promote it all the way to production without oversight from another human being.
 - CI pipeline runs automated security checks without manual effort
 - Sensitive data (secrets & tokens etc) are stored in GitHub Secrets
 - Build, test, scan and deployment stages and tasks are separated into steps and blocks
 - Self-hosted runner executes controlled pipeline tasks
 - The .yaml file responsible for the automated CI contains commented parts for real production which involves thresholds for failing the run if high or critical warning are found.
- Components from untrusted sources, across any part of the tech stack, are used in or can impact on production environments.
 - Npm is used for package installation <- gives warnings for untrusted or unmaintained technologies
 - All dependencies are installed via npm with locked versions
 - Docker images are pulled from official repositories
 - Automated testing and scanning is used to detect malicious or risky dependencies
- You do not fix or upgrade the underlying platform, frameworks, and dependencies in a risk-based, timely fashion. This commonly happens in environments when patching is a monthly or quarterly task under change control, leaving organizations open to days or months of unnecessary exposure before fixing vulnerabilities.
 - CI pipeline runs on every push and pull request which confirms continuous testing to ensure safe and working product.
 - Scanning tools detect outdated or vulnerable components instantly and the tests can be easily changed to not pass the workflow and not allow the push before fixing the errors (uncomment the threshold lines)
 - Node version is explicitly defined and selected to fit secure requirements
 - For example one Docker file contained a riskier version of Node and it was changed to use a different slim version
 - This allowed for more performant usage and reduce in unnecessary imports
 - Requires for extra install to “fill the gaps” needed, eg. python
- Software developers do not test the compatibility of updated, upgraded, or patched libraries.
 - CI pipeline installs dependencies in isolated environments for backend and frontend
 - Full test suite runs before security scanning

- Scanning tools and npm give warnings of deprecated and potentially dangerous dependencies
- Docker and Docker Compose is used to ensure versatile functioning with the set versionings
- Build steps ensure compatibility before container deployment
- Your CI/CD pipeline has weaker security than the systems it builds and deploys, especially if it is complex.
 - CI pipeline includes multiple security layers, tests and tools that are mentioned above as well as the secrets are stored in GitHub secrets and injected from there
 - Isolated Docker environment for consistency and security
 - Containers are stopped after each run

A04:2025 Cryptographic Failures

- Are any old or weak cryptographic algorithms or protocols used either by default or in older code?
 - The application implements modern cryptographic algorithms
 - Password hashing is implemented using bcrypt with 12 round salting
 - CSRF-tokens are generated with crypto using a 32-byte random value and converted to hex
 - Access and refresh token signing uses jsonwebtoken
 - 15 minute access token
 - 7 day refresh token
- Are default crypto keys in use, are weak crypto keys generated, are keys re-used, or is proper key management and rotation missing?
 - JWT signing secrets are stored as environment variables
 - No default or hardcoded cryptographic keys are used
 - Secrets are injected safely in development, production and testing modes
 - Key rotation is not implemented
- Are crypto keys checked into source code repositories?
 - They are not
 - Secrets are stored in env variables and pipeline secrets.
 - .env and database are not uploaded to GitHub
- Is encryption not enforced, e.g., are any HTTP headers (browser) security directives or headers missing?
 - Helmet is used to enforce secure HTTP headers
 - Secure cookie settings are applied (secure in production)
 - CORS is restricted to trusted origins only
 - CSRF protection is implemented using server-side session tokens
- Are initialization vectors ignored, reused, or not generated sufficiently secure for the cryptographic mode of operation? Is an insecure mode of operation such as ECB in use? Is encryption used when authenticated encryption is more appropriate?
 - No custom encryption or cryptographic algorithms are used.
 - Cryptographic operations are handled with trusted libraries
- Are passwords being used as cryptographic keys in the absence of a password based key derivation function?
 - Passwords are not used as cryptographic keys
 - Passwords are hashed using bcrypt before storage

- Is randomness used that was not designed to meet cryptographic requirements? Even if the correct function is chosen, does it need to be seeded by the developer, and if not, has the developer over-written the strong seeding functionality built into it with a seed that lacks sufficient entropy/unpredictability?
 - o Randomness used for the operations was done with secure aspect focusing libraries and not with Math.random() etc.
 - o Tokens, keys and IDs were implemented with their corresponding best seen practices (crypto.RandomBytes(), randomUUID(), bcrypt(password, salt rounds) etc)
 - o Session IDs are generated using secure session middleware
- Are cryptographic error messages or side channel information exploitable, for example in the form of padding oracle attacks?
 - o Error messages are intentionally generic to not expose any details to clients
 - o Errors are handled safely
 - o Safety related printing is done only in testing and development modes
- Can the cryptographic algorithm be downgraded or bypassed?
 - o JWT authentication with its secrets makes misuse hard since they are required for many operations. Token expiration makes this even harder
 - o CSRF protection prevents unauthorized execution using stolen cookies
 - o Secure cookies reduce token theft
 - o Algorithms are not exposed

A05:2025 Injection

- User -supplied data is not validated, filtered, or sanitized by the application.
 - o User input is validated on both client and server sides
 - o Frontend validation is done using Formik and Yup (registration rules etc for format length and complexity)
 - o Backend validation is implemented using Zod and it's done before any interaction with database
 - o Invalid input is rejected before submitting form and if invalid data would get to backend, it's also validated there
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
 - o All database operations use parametrized queries
 - Data for the queries is validated before this point is reached
- Unsanitized data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
 - o Database is used with helper functions
 - o All queries are defined and parametrized
- Potentially hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.
 - o User input is never directly used for SQL queries as they are parametrized
 - o Backend ensures all input data is passed as parameters for the query
 - o Authentication and rate limiting reduces risk of misuse

A06:2025 Insecure Design

- Establish and use a secure development lifecycle with AppSec professionals to help evaluate and design security and privacy-related controls
 - o Structured development workflow with CI/CD integration

- Security testing integrated with the tests and tools used
 - Security continuously validated on pull and push
- Establish and use a library of secure design patterns or paved-road components
 - Reusable security patterns are used
 - JWT based access token authentication middleware
 - CSRF protection middleware
 - Centralized API
 - Parametrized database actions
 - Conventional module namings and pathings and their easy usage (services, utils)
- Use threat modeling for critical parts of the application such as authentication, access control, business logic, and key flows
 - Authentication flow (login, refresh, logout)
 - Token lifecycle (access and refresh tokens with expiration)
 - Session management with Redis
 - CSRF protection
- User threat modeling as an educational tool to generate a security mindset
 - Security in layered protections (frontend validation, backend validation, middleware...)
- Integrate plausibility checks at each tier of your application (from frontend to backend)
 - Frontend validation (Formik + Yup ...)
 - Backend validation
 - Database parametrized queries
 - CSRF, and JWT token verifications
 - Docker
 - Nginx
- Write unit and integration tests to validate that all critical flows are resistant to the threat model. Compile use-cases *and* misuse-cases for each tier of your application.
 - Negative and positive test cases are written
 - Coverage tests also
 - Tool assisted also
- Segregate tier layers on the system and network layers, depending on the exposure and protection needs
 - Frontend (React + Nginx)
 - Backend (Express)
 - Database/session (SQLite + Redis)
 - Docker Compose uses private network
 - Nginx acts as a reverse proxy

A07:2025 Authentication Failures

- Permits automated attacks such as credential stuffing, where the attacker has a breached list of valid usernames and passwords. More recently this type of attack has been expanded to include hybrid password attacks credential stuffing (also known as password spray attacks), where the attacker uses variations or increments of spilled credentials to gain access, for instance trying Password1!, Password2!, Password3! and so on.
 - Rate limiting for register and login (limits the number of attempts for a certain time window, currently 10 per 15 minutes)
 - Strong password requirements make the process harder and slower
- Permits brute force or other automated, scripted attacks that are not quickly blocked.

- Login and register are protected with rate limiting as said before.
 - Invalid attempts don't leak any information, only Invalid credentials or for register Username or email already used.
- Permits default, weak, or well-known passwords, such as "Password1" or "admin" username with an "admin" password.
 - Passwords must contain 12 characters and at least one lowercase, uppercase, number and a special character.
- Allows users to create new accounts with already known-breached credentials.
 - Not implemented, although password complexity reduces risk
- Allows use of weak or ineffective credential recovery and forgot-password processes, such as "knowledge-based answers," which cannot be made safe.
 - No recovery feature
- Uses plain text, encrypted, or weakly hashed passwords data stores (see [A04:2025-Cryptographic Failures](#)).
 - Passwords are hashed using bcrypt with 12 salt rounds
 - Stored hashed
 - Verification uses secure bcrypt comparison
- Has missing or ineffective multi-factor authentication.
 - No MFA
- Allows use of weak or ineffective fallbacks if multi-factor authentication is not available.
 - No MFA
- Exposes session identifier in the URL, a hidden field, or another insecure location that is accessible to the client.
 - Session id is only stored as HTTP-only cookie sid
 - Cookies not accessible via javascript
 - Never exposed in URL or frontend storage
- Reuses the same session identifier after successful login.
 - New JWT access token issued on each login (destroyed on logout)
 - Session id is same but access tokens and CSRF change.
 - CSRF token becomes invalid once session is destroyed
- Does not correctly invalidate user sessions or authentication tokens (mainly single sign-on (SSO) tokens) during logout or a period of inactivity.
 - Application invalidates user sessions on logout by destroying the session
 - Refresh tokens are destroyed on logout
 - CSRF tokens become invalid once the session is destroyed
 - JWT access tokens expire after 15 minutes

A08:2025 Software or Data Integrity Failures

- Use digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered.
- Ensure libraries and dependencies, such as npm or Maven, are only consuming trusted repositories. If you have a higher risk profile, consider hosting an internal known-good repository that's vetted.
- Ensure that there is a review process for code and configuration changes to minimize the chance that malicious code or configuration could be introduced into your software pipeline.
- Ensure that your CI/CD pipeline has proper segregation, configuration, and access control to ensure the integrity of the code flowing through the build and deploy processes.
- Ensure that unsigned or unencrypted serialized data is not received from untrusted clients and subsequently used without some form of integrity check or digital signature to detect tampering or replay of the serialized data.

A09:2025 Security Logging and Alerting Failures

- Logging to server-side console on runtime when a key event happens (HTTP requests)
- Not currently stored anywhere ← improvement
- Errors are generic and don't leak any information but guide the user
- Handled safely for different situations

A010:2025 Mishandling of Exceptional Conditions

- Backend uses centralized error handling in Express to manage unexpected runtime exceptions
- All routes pass failures to the global error handler
- Authentication and authorization failures deny access by default
- Invalid JWT tokens or failed CSRF checks result in request rejection
- Frontend and backend validation prevent invalid data from being submitted and passed before processing
- Database operations use parametrized queries to prevent injections or malformed SQL
- Errors in database operations are handled

Testing

Manual testing as well as testing for many modules has been done. These tests are ran on every push to GitHub:

- Jest + Vitest module testing + coverage
- SAST SonarQube
- Dependency scanning Snyk
- Container scanning Trivy
- Supply chain/SBOM CycloneDX
- Runtime security/DAST OWASP ZAP

Artifacts of OWASP ZAP and CycloneDX can be found for each workflow in GitHub actions.

Module tests can be found in backend and frontend directories from directory called /tests.

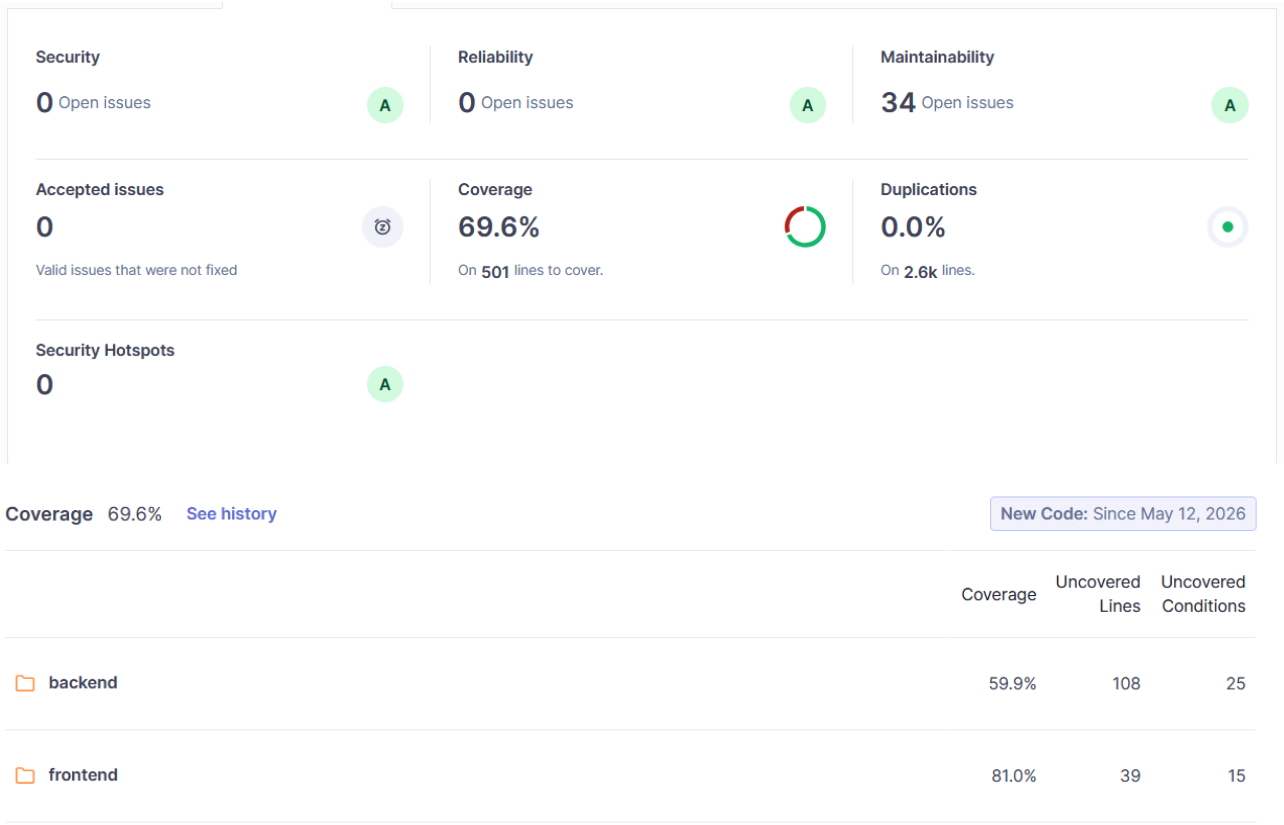
SonarQube

Security hotspots:

1. Backend and frontend Dockerfiles warning: "Copying recursively might inadvertently add sensitive data to the container. Make sure it is safe here."
 - Backend marked safe:
 - o Backend contains a dockerignore file which involves the followings which could be critical:
 - *.db files
 - .env secrets
 - git data
 - build data
 - dependencies excluded
 - Frontend fixed:
 - o Added dockerignore for frontend to ensure secure usage.
2. Deprecated property "sonar.login" replaced with "sonar.token"
3. Register page reliability issues – "4x A form label must be associated with a control" + "No duplicate props allowed"

4. High severity errors (2) in api.js – Response interceptor returns error instead of throwing them – fixed

This is what SonarQube statistics look like. Coverage doesn't reach 80% since some of the parts are super hard and time consuming to test. Also using the log() reduces the coverage since it's not tested.



Issues it shows look something like this.

☐ Prefer `'node:crypto'` over `'crypto'`.

Consistency

Maintainability Medium

convention import ... +

Open Not assigned

L1 • 5min effort • 5 days ago

backend/utils/validation.js

☐ Use concise character class syntax `'\w'` instead of `'[a-zA-Z0-9_]'`.

Intentionality

Maintainability Low

regex type-dependent +

Open Not assigned

L21 • 5min effort • 8 days ago

☐ `'(params?: string | { abort?: boolean; pattern?: RegExp; error?: string | $ZodErrorMap<$ZodIssueInvalidStringFormat>; message?: string; })':` `ZodString` is deprecated.

Consistency

Maintainability Low

cwe type-dependent ... +

Open Not assigned

L23 • 15min effort • 8 days ago

Snyk

Security hotspots:

1. Frontend Dockerfile nginx version changed to more stable one
2. Backend Dockerfile noder version changed to more secure one

	BASE IMAGE	VULNERABILITIES	SEVERITY
Current image	nginx:1.27-alpine	57	3 C 13 H 5 M 36 L
Minor upgrades	nginx:1.30.0-alpine	3	0 C 0 H 1 M 2 L

	BASE IMAGE	VULNERABILITIES	SEVERITY
Current image	node:20-slim	73	3 C 3 H 2 M 65 L
Major upgrades	node:26.1.0-slim	43	0 C 0 H 1 M 42 L

I believe Snyk also pointed out that “The "node" image runs with "root" as the default user. Make sure it is safe here.” This was fixed by running node image as non-root-user and adjusting write/read/execute permissions.

ZAP OWASP

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	4
Low	6
Informational	3
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	4
HTTP Only Site	Medium	1
Missing Anti-clickjacking Header	Medium	4
Relative Path Confusion	Medium	3
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	4
Permissions Policy Header Not Set	Low	5
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5
X-Content-Type-Options Header Missing	Low	5
Information Disclosure - Suspicious Comments	Informational	1
Modern Web Application	Informational	4
Storable and Cacheable Content	Informational	5

No high vulnerabilities. Found vulnerabilities:

1. Missing security headers
 - CSP header missing
 - Fix: add CSP header
 - Anti-clickjacking header missing
 - Fix X-FRAME-OPTIONS: DENY or Content-Security-Policy: frame-ancestors 'none';
2. Server info leakage
 - Server Leaks Version Information via "Server" HTTP Response Header Field (Server: nginx/1.30.0)
 - Hide server version

3. HTTP-only site
 - Would be HTTPS in production
4. Relative path confusion
 - To my understanding an unusual edge-case browser parsing issue
5. Missing CSP extensions, X-Content-Type-Options
 - Add CSP-Policy
 - Add frame protection headers
 - Adjust Express, CORS and helmet.js configurations

Trivy frontend and backend image scans

```
secure-auth-app-frontend (alpine 3.23.4)
=====
Total: 3 (UNKNOWN: 0, LOW: 0, MEDIUM: 2, HIGH: 1, CRITICAL: 0)
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
libxpm	CVE-2026-4367	MEDIUM	fixed	3.5.17-r0	3.5.19-r0	libXpm Out-of-bounds read in xpmNextWord() https://avd.aquasec.com/nvd/cve-2026-4367
nghttp2-libs	CVE-2026-27135	HIGH		1.68.0-r0	1.68.1	nghttp2: nghttp2: Denial of Service via malformed HTTP/2 frames after session termination... https://avd.aquasec.com/nvd/cve-2026-27135
xz-libs	CVE-2026-34743	MEDIUM		5.8.2-r0	5.8.3-r0	xz: XZ Utils: Denial of Service via buffer overflow in index decoding... https://avd.aquasec.com/nvd/cve-2026-34743

```
secure-auth-app-backend (debian 13.4)
=====
Total: 978 (UNKNOWN: 13, LOW: 590, MEDIUM: 308, HIGH: 67, CRITICAL: 0)
```

Smaller base image could help this. One other option could be updating OS. With some parameters we can try to see high and critical errors which have been marked with a valid fix. This shows a very different result.

```
1 ▶ Run trivy image secure-auth-app-backend --severity HIGH,CRITICAL --ignore-unfixed
4 2026-05-15T17:21:34+03:00 INFO [vuln] Vulnerability scanning is enabled
5 2026-05-15T17:21:34+03:00 INFO [secret] Secret scanning is enabled
6 2026-05-15T17:21:34+03:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
7 2026-05-15T17:21:34+03:00 INFO [secret] Please see https://trivy.dev/docs/v0.70/guide/scanner/secret#recommendation for
8 2026-05-15T17:21:42+03:00 INFO Detected OS family="debian" version="13.4"
9 2026-05-15T17:21:42+03:00 INFO [debian] Detecting vulnerabilities... os_version="13" pkg_num=140
10 2026-05-15T17:21:43+03:00 INFO Number of language-specific files num=1
11 2026-05-15T17:21:43+03:00 INFO [node-pkg] Detecting vulnerabilities...
12 2026-05-15T17:21:43+03:00 WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/vulnerability#severity-selection for details.
```

13

14 Report Summary

15

Target	Type	Vulnerabilities	Secrets
secure-auth-app-backend (debian 13.4)	debian	0	-

19

Non-implemented and questionable parts

I would've liked to put some material for each user that they could've fetched from MainPage but I didn't have time for that. Some sort of material creation eg. uploading a picture or creating a text file could be something like this, and one could select to "share" it for other people and only people who are assigned content can access and view it.

Logging is done runtime only, safely on development mode but not written anywhere. This would be a necessity for a real secure system and thus would require adding in the future. This would involve for example logging access control failures, alert when failures are repeated, suspicious activity, unexpected errors as well as normal activity.

API interceptors were used for Axios communication for communication. I thought I read something bad about them and not to use them but now that I read again from OWASP CSRF prevention cheat sheet, they are talked almost as a necessity. Thus, I do not know if they are good or bad to use in this scenario since I've thought I should've implemented the solution some other way but could not get working without using them.

OWASP talks about MFA, password/user credential recovery and both together and my application lacks them both. Future updates would be to apply MFA and credential recovery. Sending a confirmation email of user registration would also be great since now anyone could make users for email they don't have access to. Credential recovery could also go through email.

My "check token rotations and token refresh options" tasks are highlighted on to-do list. Functions and logic for them have already been implemented, but I'm not sure of their real functioning. After 15 minutes, the user session is not refreshed and trying to logout results in unauthorized logout while user is really logged out from the server-side, the frontend does fallback cleanup upon logout. Having some other features on the site would help to debug this since logging out could be implemented in separate ways. Right now expired token returns 403 and API interceptor doesn't respond to 403 by trying to refresh the token. This should be changed to match the status and actions. Also smarter logout implementation would be nice. Right now if logout fails the browser creates JWT-token and XSRF-tokens which are unnecessary and could be potentially dangerous. Also the App could be adjusted to try to login the user if there's still a valid session ongoing in case the user enters a different site without logging on, since the cookies still exist in the storage.

More excessive testing would be ideal. In order to verify proper functioning of code thus securing the software broader testing should be implemented. This involves higher coverage (testing all code files – the more greater impact and importance to the software, the more tests should be targeted towards a certain file. These involve edge and limit case testing, positive and negative test cases as well as testing different error situations, the list is long. Following at least:

- Proper functioning
- Right typing
- Edge cases handled
- Unexpected situation/error handling
- End-to-end tests which are added to CI to secure proper functioning on every push!
- Splitting one pipeline into multiple ones to enable easier and faster runs on smaller updates. Now the CI/CD takes about 8-12 minutes per run so having smaller tests on dev branch with some selected CI pipeline with casual fast tests would be beneficial for the project. For example spending time on backend npm installation on CI after pushing an update regarding frontend isn't that necessary.

- setup test environment where there's no mocks etc but real scenario simulations including real database
- Integration testing, testing without mocks and actual testing databases (with testing environment!)
- Taking advantage of the CI technologies and their feedback! For the sake of this project, only examining was done on most parts and fixing/changing was done on higher criticality related threats. Noticing and acknowledging the vulnerabilities/threats/alerts is super important in secure programming! One upgrade to the program would be to fix most of the problems, especially ones reported by OWASP ZAP.

AI Usage:

- Docker Compose and Docker file creation
 - o AI was prompted "Im gonna build a docker compose can you help me construct docker compose as well as dockerfiles for backend and frontend". I then gave out information of technologies used and the AI gave me templates which I then adjusted to work and suit best for my usage.
- Testing
 - o AI was given files and asked "can you make me tests for this file". Depending on if the file was frontend or backend, it was told to use vitest or jest. These answers were then used as a template to make testing faster. Some files required just basic testing due to their simple logic and responsibilities and AI answers came in handy since some of them only required little editing and reviewing. More advanced testing files took AI response in mind, but AI mostly focuses on positive testing. These were edited and reviewed, and negative tests are mainly done by hand.
- CI pipeline .yaml file
 - o Some errors regarding to pipeline in GitHub actions as well as self-runner setup were done with googling and AI assistance. It was asked many times "what is this error, how do I fix it, here is my current .yaml file". It then gave explanations as well as options on how to fix the problem. These were then selected out by their best fitting in the program and sometimes tested which works the best and then proceeded customizing the AI response.
- General debugging
 - o AI has been used to assist debugging program flow, console logs and errors. For example, when I had an authentication access token problem, I reviewed my code multiple times without finding out why exactly my code was failing. I then told AI how my code was supposed to work, snippets of the parts
- Feedback and improvements on source code
 - o I gave AI my code files, told what they were supposed to do and what were their purpose and asked its opinions and some of its answers were used as improvements. These involved redundant code lines and imports left unnoticed as well as actual code improvements on the API interceptors and CORS setup. Some smaller conditional rules were also adjusted after AI notification due to accidentally having reversed condition etc.